

【Java 试题总结4】JavaWeb 、 络、设计模式综合篇（附答案）

Java Web 篇

一、jsp 和 servlet有什么区别？

- 1、servlet是服务器端的Java程序，它担当客户端和服务端的中间层。
- 2、jsp全名为Java server pages 中 名叫Java服务器页 ，其本质是 一个简化的servlet设计。JSP 是 一种动态页 设计，它的主要 的是将表 逻辑从servlet中分离出来。
- 3、JVM 只能识别Java代码，不能识别JSP ， JSP 编译后变成了servlet web 容器将JSP 的代码编译成JVM 能够识别的Java类（servlet）。
- 4、JSP 有内置对象、servlet没有内置对象。

二、jsp 有哪些内置对象？作 分别是什么？

JSP 九 内置对象：

1. pageContext，页 上下 对象，相当于页 中所有功能的集合，通过它可以获取JSP 页 的out、request、response、session、application对象。
2. request
3. response
4. session
5. application 应 程序对象，application实现了 户间数据的共享，可存放全局变量，它开始于服务器启动，知道服务器关闭。
6. page，就是JSP 本 。
7. exception
8. out，out 于在web 浏览器内输出信息，并且管理应 服务器上的输出缓冲区，作 域page 。
9. config 取得服务器的配置信息。

三、forward 和 redirect的区别？

1. forward是直接请求转发；redirect是间接请求转发， 叫重定向。
2. forward，客户端和浏览器执 次请求；redirect 客户端和浏览器执 两次请求。
3. forward，经典的MVC 模式就是forward；redirect 于避免 户 的正常访问。（例如 户 正常访问，servlet就可以将HTTP 请求重定向到登录页 ）。
4. forward，地址不变；redirect 地址改变。
5. forward常 法：RequestDispatcher类的forward() 法；redirect常 法：HttpServletResponse类的sendRedirect() 法。

四、说 下 jsp的 4种作 域？

application session、request、page

五、session 和 cookie 有什么区别？

- 1、存储位置不同
 - cookie在客户端浏览器；
 - session在服务器；
- 2、存储容量不同
 - cookie<=4K ， 一个站点最多保留20个cookie；
 - session没有上线，出于对服务器的保护，session内不可存过多东西，并且要设置session删除机制；
- 3、存储 式不同
 - cookie只能保存ASCII字符串，并需要通过编码 式存储为Unicode 字符或者 进制数据；
 - session中能存储任何类型的数据，包括并不局限于String integer list map 等；
- 4、隐私策略不同
 - cookie对客户端是可见的，不安全；
 - session存储在服务器上，安全；

5、有效期不同

- 开发可以通过设置cookie的属性，达到使cookie长期有效的效果；
- session依赖于名为JSESSIONID 的cookie， cookie JSESSIONID 的过期时间默认为-1，只需关闭窗 该session就会失效，因 session达不到长期有效的效果；

6、跨域 持上不同

- cookie 持跨域；
- session不 持跨域；

六、如果客户端禁 cookie 能实现 session 还能 吗？

般默认情况下，在会话中，服务器存储 session的 sessionId是通过 cookie存到浏览器 。

如果浏览器禁 了 cookie 浏览器请求服务器 法携带 sessionId 服务器 法识别请求中的 户 份，session失效。

但是可以通过其他 法在禁 cookie的情况下，可以继续使 session。

1. 通过ur重写，把 sessionId作为参数追加的原 ur中，后续的浏览器与服务器交互中携带 sessionId参数。
2. 服务器的返回数据中包含 sessionId 浏览器发送请求时，携带 sessionId参数。
3. 通过 Http协议其他 header 字段，服务器每次返回时设置该 header 字段信息，浏览器中 js读取该 header 字段，请求服务器时，js设置携带该 header 字段。

七、什么是上下 切换？

多线程编程中 般线程的个数都 于 CPU 核 的个数， 个 CPU 核 在任意时刻只能被 个线程使 ，为了让这些线程都能得到有效执 ，CPU 采取的策略是为每个线程分配时间 并轮转的形式。当 个线程的时间 完的时候就会重新处于就绪状态让给其他线程使 ，这个过程就属于 次上下 切换。

概括来说就是：当前任务在执 完 CPU 时间 切换到另 个任务之前会先保存 的状态，以便下次再切换回这个任务时，可以再加载这个任务的状态。任务从保存到再加载的过程就是 次上下 切换。

上下 切换通常是计算密集型的。也就是说，它需要相当可观的处理器时间，在每秒 上百次的切换中，每次切换都需要纳秒量级的时间。所以，上下 切换对系统来说意味着消耗 量的 CPU 时间，事实上，可能是操作系统中时间消耗最 的操作。

Linux相 与其他操作系统（包括其他类 Unix系统）有很多的优点，其中有 项就是，其上下 切换和模式切换的时间消耗 常少。

、cookie、session、token

1、session机制

session是服务端存储的 个对象，主要 来存储所有访问过该服务端的客户端的 户信息（也可以存储其他信息），从 实现保持 户会话状态。但是服务器重启时，内存会被销毁，存储的 户信息也就消失了。

不同的 户访问服务端的时候会在session对象中存储键值对， 键” 来存储开启这个 户信息的 钥匙”；在登录成功后， 钥匙”通过cookie返回给客户端，客户端存储为sessionId记录在cookie中。当客户端再次访问时，会默认携带cookie中的sessionId来实现会话机制。

(1) session是基于cookie的。

- cookie的数据4k左右；
- cookie存储数据的格式：字符串key=value
- cookie存储有效期：可以 通过expires进 具体的 期设置，如果没设置，默认是关闭浏览器时失效。
- cookie有效范围：当前域名下有效。所以session这种会话存储 式 式只适 于客户端代码和服务端代码运 在同 台服务器上（前后端项 协议、域名、端 号都 致，即在 个项 下）

(2) session持久化

于解决重启服务器后session消失的问题。在数据库中存储session， 不是存储在内存中。通过包：express-mysql-session

当客户端存储的cookie失效后，服务端的session不会 即销毁，会有 个延时，服务端会定期清理 效session，不会造成 效数据占 存储空间的问题。

2、token机制



适用于前后端分离的项目（前后端代码运行在不同的服务器下）

请求登录时，token和sessionid原理相同，是对key和key对应的用户信息进行加密后的加密字符，登录成功后，会在响应主体中将{token: "字符串"}返回给客户端。

客户端通过cookie都可以进行存储。再次请求时不会默认携带，需要在请求拦截器位置给请求头中添加认证字段Authorization携带token信息，服务器就可以通过token信息查找用户登录状态。

九、说一下 session 的工作原理？

当客户端登录完成后，会在服务端产生一个session，此时服务端会将sessionid返回给客户端浏览器。客户端将sessionid储存在浏览器的cookie中，当用户再次登录时，会获得对应的sessionid 然后将sessionid发送到服务端请求登录，服务端在内存中找到对应的sessionid完成登录，如果找不到，返回登录页。

络篇

一、http 响应码 301 和 302 代表的是什么？有什么区别？

- 1. 301 和302 状态码都表示重定向，当浏览器拿到服务器返回的这个状态码后会自动跳转到一个新的URL 地址。
- 2. 301 代表永久性重定向，旧地址被永久移除，客户端向新地址发送请求。
- 3. 302 代表暂时性重定向，旧地址还在，客户端继续向旧地址发送请求。
- 4. 303 代表暂时性重定向，重定向到新地址时，必须使用 GET 方法请求新地址。
- 5. 307 代表暂时性重定向，与302 的区别在于307 不允许从POST 改为GET 。
- 6. 308 代表永久性重定向，与301 的区别在于308 不允许从POST 改为GET 。

二、简述 tcp和 udp的区别？

- 1. TCP 是传输控制协议，UDP 是用户数据报协议；
- 2. TCP 长连接，UDP 短连接；
- 3. UDP 程序结构较简单，只需发送，无须接收；
- 4. TCP 可靠，保证数据正确性、顺序性；UDP 不可靠，可能丢数据；
- 5. TCP 适用于少量数据，UDP 适用于大量数据传输；
- 6. TCP 速度慢，UDP 速度快；

三、tcp 为什么要三次握手，两次不行吗？为什么？

因为客户端和服务端都要确认连接，①客户端请求连接服务端；②针对客户端的请求确认应答，并请求建立连接；③针对服务端的请求确认应答，建立连接；

两次无法确保A 能收到B 的数据；

四、OSI 的七层模型都有哪些？

五、get 和 post 请求有哪些区别？

- 1. get请求参数是连接在url后的， post请求参数是存放在requestbody内的；
- 2. get请求因为浏览器对url长度有限制，所以参数个数有限制， post请求参数个数没有限制；
- 3. 因为get请求参数暴露在url上,所以安全 post get更加安全；
- 4. get请求只能进 url编码， post请求可以 持多种编码 式；
- 5. get请求参数会保存在浏览器历史记录内, pos请求并不会；
- 6. get请求浏览器会主动cache, post并不会,除 主动设置；
- 7. get请求产 1个tcp数据包, pos请求产 2个tcp数据包；
- 8. 在浏览器进 回退操作时, get请求是 害的， post请求则会重新请求 次；
- 9. 浏览器在发送get请求时会将header和data 起发送给服务器,服务器返回200 状态码， 在发送post请求时,会先将header发送给服务器, 服务器返回100,之后再将data发送给服务器,服务器返回200 OK ；

六、什么是 XSS 攻击，如何避免？

xss(Cross Site Scripting即跨站脚本攻击，是 种常见于web 应 程序中的计算机安全漏洞。指的是在 户浏览器上，在渲染DOM 树的时候，执 了不可预期的JS 脚本，从 发 了安全问题。

XSS 就是通过在 户端注 恶意的可运 脚本，若服务端对 户的输 不进 处理，直接将 户的输 输出到浏览器，然后浏览器将会执 户注 的脚本。所以XSS 攻击的核 就是浏览器渲染DOM 的时候将 本信息解析成JS 脚本从 引发JS 脚本注 ，那么XSS 攻击的防御段就是基于浏览器渲染这 步去做防御。只要我们使 HTML 编码将浏览器需要渲染的信息编码后，浏览器在渲染DOM 元素的时候，会 动解码需要渲染的信息，将上述信息解析成字符串 不是JS 脚本，这就是我们防御XSS 攻击的核 想法。

- 预防：
- 1、获取 户的输 ，不 innerHtml, innerText.
 - 2、对 户的输 进 过滤，如对& < > " 等进 转义；

七、什么是 CSRF 攻击，如何避免？

跨站请求伪造（英语：Cross-site request forgery，也被称为 one-click attack或者 session riding通常缩写为 CSRF 或者 XSRF ，是种挟制 户在当前已登录的Web 应 程序上执 本意的操作的攻击 法。跟跨 站脚本（XSS ）相 ，XSS 利 的是 户对指定 站的信任，CSRF 利 的是 站对 户 页浏览器的信任。

1、攻击细节

跨站请求攻击，简单地说，是攻击者通过 些技术 段欺骗 户的浏览器去访问 个 曾经认证过的 站并运 些操作（如发邮件，发消息，甚 财产操作如转账和购买商品）。由于浏览器曾经认证过，所以被访问的 站会认为是真正的 户操作 去运 。这利 了web 中 户 份验证的 个漏洞：简单的 份验证只能保证请求发 某个 户的浏览器，却不能保证请求本 是 户 愿发出的。

例

那么， 个恶意攻击者可以在另 个 站上放置如下代码：

如果有账户名为Alice的 户访问了恶意站点， 她之前刚访问过银 不久，登录信息尚未过期，那么她就会损失1000 资 。

这种恶意的 址可以有很多种形式，藏 于 页中的许多地 。此外，攻击者也不需要控制放置恶意 址的 站。例如他可以将这种地址藏在论坛，博客等任何 户 成信息的 站中。这意味着如果服务端没有合适的防御措施的话， 户即使访问熟悉的可信 站也有受攻击的危险。

透过例 能够看出，攻击者并不能通过CSRF 攻击来直接获取 户的账户控制权，也不能直接窃取 户的任何信息。他们能做到的，是欺骗 户浏览器，让其以 户的名义运 操作。

2、防御措施

检查Referer字段

HTTP 头中有一个Referer字段，这个字段 以标明请求来源于哪个地址。在处理敏感数据请求时，通常来说，Referer字段应和请求的地址位于同 域名下。以上 银 操作为例，Referer字段地址通常应该是转账按钮所在的 页地址，应该也位于 之下。如果是CSRF 攻击传来的请求，Referer字段会是包含恶意 址的地址，不会位于 之下，这时候服务器就能识别出恶意的访问。

这种办法简单易 ， 作量低，仅需要在关键访问处增加 步校验。但这种办法也有其局限性，因其完全依赖浏览器发送正确的Referer字段。虽然http协议对此字段的内容有明确的规定，但并 法保证来访的浏览器的具体实现，亦 法保证浏览器没有安全漏洞影响到此字段。并且也存在攻击者攻击某些浏览器，篡改其Referer字段的可能。

3、添加校验token

由于CSRF 的本质在于攻击者欺骗 户去访问 设置的地址，所以如果要求在访问敏感数据请求时，要求 户浏览器提供不保存在cookie 中，并且攻击者 法伪造的数据作为校验，那么攻击者就 法再运 CSRF 攻击。这种数据通常是窗体中的 个数据项。服务器将其 成并

附加在窗体中，其内容是一个伪随机数。当客户端通过窗体提交请求时，这个伪随机数也一并提交上去以供校验。正常的访问时，客户端浏览器能够正确得到并传回这个伪随机数，通过CSRF传来的欺骗性攻击中，攻击者从事先得知这个伪随机数的值，服务端就会因为校验token的值为空或者错误，拒绝这个可疑请求。

八、如何实现跨域？说一下JSONP实现原理？

- 1、
- 2、最流行的跨域方案cors

cors是目前主流的跨域解决方案，跨域资源共享(CORS)是一种机制，它使额外的HTTP头来告诉浏览器让运行在一个origin(domain)上的Web应用被准许访问来自不同源服务器上的指定的资源。当一个资源从与该资源本身所在的服务器不同的域、协议或端口请求一个资源时，资源会发起一个跨域HTTP请求。

- 3、最简便的跨域方案Nginx

nginx是一款极其强大的web服务器，其优点就是轻量级、启动快、并发。

现在的新项目中nginx几乎是首选，我们node或者java开发的服务通常都需要经过nginx的反向代理。

反向代理的原理很简单，即所有客户端的请求都必须先经过nginx的处理，nginx作为代理服务器再讲请求转发给node或者java服务，这样就规避了同源策略。

九、websocket应用的是哪个协议

WebSocket是一个允许Web应用程序(通常指浏览器)与服务器进行双向通信的协议。HTML5的WebSocket API主要是为浏览器端提供了一个基于TCP协议实现全双通信的方法。

WebSocket优势：浏览器和服务器只需要要做个握手的动作，在建连接之后，双方可以在任意时刻，相互推送信息。同时，服务器与客户端之间交换的头信息很。

十、说一下tcp粘包是怎么产生的？

- 1. 发送需要等缓冲区满才能发送出去，造成粘包；
- 2. 接收不及时接收缓冲区的包，造成粘包；

设计模式篇

一、请列举出在JDK中一个常见的设计模式？

- 1、单例模式

作用：保证类只有一个实例。

JDK中体现：Runtime类。

- 2、静态工厂模式

作用：代替构造函数创建对象，方法名构造函数清晰。

JDK中体现：Integer.valueOf Class.forName

- 3、抽象

作用：创建某种类的对象。

JDK中体现：Java.sql包。

- 4、原型模式

clone()

原型模式的本质是拷贝原型来创建新的对象，拷贝是new更快的创建对象的方法，当需要批量创建新对象且都是同一个类的对象的时候考虑使用原型模式。

一般的克隆只是浅拷贝（对象的hash值不一样，但是对象的成员变量的hash值是一样的）。

有些场景需要深拷贝，这时我们就要重写clone方法，以ArrayList为例：

5、适配器模式

作：使不兼容的接口相容。

JDK 中体现：InputStream、OutputStream。

6、装饰器模式

作：为类添加新的功能，防止类继承带来的类爆炸。

JDK 中体现：io类、Collections List

7、外观模式

作：封装一组交互类，直接对外提供接口。

JDK 中体现：logging包。

8、享元模式

作：共享对象、节省内存。

JDK 中体现：Integer.valueOf String常量池。

9、代理模式

作：

- （1）透明调用被代理对象，无须知道复杂实现细节；
- （2）增加被代理类的功能；

JDK 中体现：动态代理。

10、迭代器模式

作：将集合的迭代和集合本身分离。

JDK 中体现：Iterator

11、命令模式

作：封装操作，使接口一致。

JDK 中体现：Runnable、Callable ThreadPoolExecutor。

二、什么是设计模式？你是否在你的代码中使用过任何设计模式？

1、什么是设计模式？

设计模式是解决软件开发某些特定问题提出的一些解决方案，也可以理解为解决问题的一些固定思路。

通过设计模式可以帮助我们增强代码的可复用性、可扩展性、灵活性。

我们使用设计模式的最终目的是实现代码的内聚、低耦合。

2、设计模式的七大原则

- 1. 单一职责原则
- 2. 接口隔离原则
- 3. 依赖倒转原则
- 4. 模式替换原则
- 5. 开闭原则
- 6. 迪米特法则
- 7. 合成复用原则

3、你是否在你的代码中使用过任何设计模式？

- （1）单例模式

JDK 中的runtime, Spring中的singeton。

- （2）简单工厂模式

Spring的BeanFactory，根据传入一个唯一标识来获得bean对象。

（3）原型模式

clone()

（4）代理模式

Spring的AOP 中，Spring实现AOP 功能的原理就是代理模式，①JDK 动态代理。②CGLIB 动态代理，使 Advice（通知）对类进行方法级别的切面增强。

（5）装饰器模式

为类添加新的功能，防止类爆炸；

IO流、数据源包装，Spring中用到的装饰器模式表现在Wrapper 。

三、Java 中什么叫单例设计模式？请写出线程安全的单例模式

1. 保证程序只有一个对象的实例，叫做单例模式；
2. 内部类的方式实现单例模式，是线程安全的；
3. 双重验证方式实现单例模式也是线程安全的；

四、在 Java 中，什么叫观察者设计模式（observer design pattern）？

1、观察者模式是一种对多的依赖关系，让多个观察者同时监听某主题对象。当这个主题对象发生变化时，会通知所有观察者对象，使它们能够自动更新。

2、JAVA 提供的对观察者模式的支持

在JAVA 语言的java.util.包中，提供了一个Observable类以及一个Observer接口，构成JAVA 语言对观察者模式的支持。

（1）Observer接口

这个接口只定义了一个方法，即update() 方法，当被观察者对象的状态发生变化时，被观察者对象的notifyObservers() 方法就会调用这个方法。

```
public interface Observer {  
    void update(Observable o, Object arg);  
}
```

（2）Observable类

被观察者类都是java.util.Observable类的子类。java.util.Observable提供公开的方法支持观察者对象，这些方法中有两个对Observable的类很重要：一个是setChanged()，另一个是notifyObservers() 方法setChanged()被调用之后会设置一个内部标记变量，代表被观察者对象的状态发生了变化。另一个是notifyObservers() 这个方法被调用时，会调用所有登记过的观察者对象的update() 方法，使这些观察者对象可以更新。

五、使用观察者模式最主要的好处是什么？在哪里使用？

1、观察者模式好处

- 良好的封装性、代码结构清晰；
- 扩展性好，如果想增加一个产品，只需扩展一个类即可；
- 典型的解耦框架；

2、在哪里使用？

- 需要成对象的地方；
- 不同数据库的访问；

六、请解释 动态装配模式的区别？

有五种 动态装配的方式，可以用来指导 Spring容器 动态装配 式来进行依赖注入。

1、no

默认的方式是不进行 动态装配，通过显式设置 re属性来进行 装配。第 402 页 共 485 页

2、byName

通过参数名 动态装配，Spring 容器在配置文件中发现 bean

的 autowire属性被设置成 byname ，之后容器试图匹配、装配和该 bean 的属性

性具有相同名字的 bean。

3、byType:

通过参数类型 动装配，Spring 容器在配置 件中发现 bean 的 autowire属性被设置成 byType，之后容器试图匹配、装配和该 bean 的属性具有相同类型的 bean。如果有多个 bean 符合条件，则抛出错误。

4、constructor

这个 式类似于 byType， 但是要提供给构造器参数，如果没有确定的带参数的构造器参数类型，将会抛出异常。

5、autodetect

先尝试使 constructo来 动装配，如果 法 作，则使 byType 式。

七、举 个 Java实现的装饰模式(decorator design pattern它是作 于对象层次还是类层次？在Java IO中运 了装饰器模式，inputStream作为抽象类，其下有 个实现类，表 从不同的数据源输 ：

- 1. byteArrayInputStream
- 2. fileInputStream
- 3. StringBufferInputStream
- 4. PipedInputStream，从管道产 输 ；
- 5. SequenceInputStream ，可将其他流收集合并到 个流内；

FilterInputStream作为装饰器在JDK 中是 个普通类，其下 有多个具体装饰器 如BufferedInputStream DataInputStream等。

FilterInputStream内部封装了基础构件：

```
protected volatile InputStream in;
```

BufferedInputStream在调 其read()读取数据时会委托基础构件来进 更底层的操作， 它 所起的装饰作 就是缓冲，在源码中可以很清楚的看到这 切。

往期精彩内容：